

A01-Vector-Embeddings-Standards

1 Vector Embeddings Standards

Document Version: 1.0

Generated: December 4, 2025

Standard: Vector Embedding Format Specifications

Status: Community/Industry Standards

1.1 Overview

Vector embeddings are numerical representations of semantic concepts. Standardization around embedding formats, dimensions, and distance metrics is critical for:

- Interoperability:** Moving embeddings between systems
- Performance:** Optimized storage and computation
- Compatibility:** Working with vector databases and retrieval systems

1.1.1 Key Concepts

- Dimensionality:** Number of values per embedding
- Data Type:** Float32, Float16, Int8 quantization
- Normalization:** L2, L1, or no normalization
- Distance Metrics:** Cosine, Euclidean, dot product
- Scale Factor:** Range of values

1.2 Authoritative References

- Hugging Face Embeddings:** <https://huggingface.co/docs/hub/models-embedding-models>
- Sentence Transformers:** <https://www.sbert.net>
- OpenAI Embeddings:** <https://platform.openai.com/docs/guides/embeddings>
- Vector Search Standards:** <https://github.com/facebookresearch/faiss>

1.3 Format Structure

1.3.1 Standard Embedding Dimensions

Model Type	Dimension	Primary Use	Latency	Accuracy
Lightweight	384	Mobile, edge	< 10ms	0.75-0.80
Standard	768	General purpose	10-50ms	0.85-0.90
Large	1024	High accuracy	50-100ms	0.90-0.95
XLarge	1536	OpenAI, Cohere	100-300ms	0.95+
Multilingual	512-768	Cross-lingual	10-50ms	0.80-0.88

1.3.2 Embedding Data Structure

Python Example (NumPy):

```
import numpy as np

# Single embedding (vector)
embedding = np.array([0.123, -0.456, 0.789, ...], dtype=np.float32)
# Shape: (768,)

# Batch of embeddings
embeddings_batch = np.array([
    [0.123, -0.456, 0.789, ...],
    [0.234, -0.567, 0.890, ...],
    [0.345, -0.678, 0.901, ...]
], dtype=np.float32)
# Shape: (3, 768)
```

JSON Format (REST API):

```
{
  "data": [
    {
      "object": "embedding",
      "embedding": [0.123, -0.456, 0.789, ...],
      "index": 0
    }
  ],
  "model": "text-embedding-3-large",
  "usage": {
    "prompt_tokens": 100,
    "total_tokens": 100
  }
}
```

Protobuf Format:

```
message Embedding {
  repeated float values = 1;
  int32 dimension = 2;
  string model_name = 3;
  int64 timestamp = 4;
}

message EmbeddingBatch {
  repeated Embedding embeddings = 1;
  int32 batch_size = 2;
}
```

1.4 Distance Metrics

1.4.1 Cosine Similarity

Formula (normalized vectors):

$$\text{similarity} = \vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$$

Properties: - Range: [-1, 1] for normalized vectors - Invariant to magnitude - Ideal for semantic similarity - Efficient computation: O(n)

Python Implementation:

```

import numpy as np

def cosine_similarity(v1: np.ndarray, v2: np.ndarray) -> float:
    """Compute cosine similarity between normalized vectors"""
    # Assume vectors already normalized (L2)
    return np.dot(v1, v2)

def cosine_similarity_unnormalized(v1: np.ndarray, v2: np.ndarray) -> float:
    """Cosine similarity with normalization"""
    norm_v1 = v1 / np.linalg.norm(v1)
    norm_v2 = v2 / np.linalg.norm(v2)
    return np.dot(norm_v1, norm_v2)

# Batch computation (efficient)
def cosine_similarity_batch(query: np.ndarray, corpus: np.ndarray) -> np.ndarray:
    """
    Args:
        query: shape (d,)
        corpus: shape (n, d)
    Returns:
        scores: shape (n,)
    """
    # Normalize corpus rows
    corpus_normalized = corpus / np.linalg.norm(corpus, axis=1, keepdims=True)
    query_normalized = query / np.linalg.norm(query)

    # Batch dot product
    scores = np.dot(corpus_normalized, query_normalized)
    return scores

```

1.4.2 Euclidean Distance

Formula:

$$d = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

Properties: - Range: $[0, \infty)$ - Affected by magnitude - Suitable for dense clusters - Computation: $O(n)$

C++ Implementation:

```

#include <cmath>
#include <vector>

float euclidean_distance(
    const std::vector<float>& v1,
    const std::vector<float>& v2) {

    float sum_sq = 0.0f;
    for (size_t i = 0; i < v1.size(); ++i) {
        float diff = v1[i] - v2[i];
        sum_sq += diff * diff;
    }
    return std::sqrt(sum_sq);
}

// Squared euclidean (faster, preserves ordering)
float euclidean_distance_squared(
    const std::vector<float>& v1,
    const std::vector<float>& v2) {

    float sum_sq = 0.0f;
    for (size_t i = 0; i < v1.size(); ++i) {
        float diff = v1[i] - v2[i];
        sum_sq += diff * diff;
    }
    return sum_sq;
}

```

1.4.3 Dot Product

Formula:

$$\text{similarity} = \vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$$

Properties: - Range: $[-\infty, \infty]$ - Requires normalized vectors for cosine-like behavior - Fastest metric (no division) - Used in maximum inner product search (MIPS)

Usage:

```

# Fast batch dot product
def dot_product_similarity(query: np.ndarray, corpus: np.ndarray) -> np.ndarray:
    """Dot product similarity"""
    # BLAS optimized
    return np.dot(corpus, query)

```

1.5 Normalization Standards

1.5.1 L2 Normalization (Most Common)

Formula:

$$\vec{v}_{norm} = \frac{\vec{v}}{\|\vec{v}\|_2} = \frac{\vec{v}}{\sqrt{\sum_{i=1}^n v_i^2}}$$

Properties: - Resultant vector has unit norm (magnitude = 1) - Used by most modern embedding models - Enables cosine similarity = dot product - Range: $[-1, 1]$ per dimension

Implementation:

```

import numpy as np

def normalize_l2(embeddings: np.ndarray) -> np.ndarray:
    """L2 normalization"""
    if len(embeddings.shape) == 1:
        # Single embedding
        return embeddings / np.linalg.norm(embeddings)
    else:
        # Batch of embeddings
        norms = np.linalg.norm(embeddings, axis=-1, keepdims=True)
        return embeddings / norms

# Example
emb = np.array([3.0, 4.0])
normalized = normalize_l2(emb)
print(normalized) # [0.6, 0.8]
print(np.linalg.norm(normalized)) # 1.0

```

1.5.2 L1 Normalization

Formula:

$$\vec{v}_{norm} = \frac{\vec{v}}{\|\vec{v}\|_1} = \frac{\vec{v}}{\sum_{i=1}^n |v_i|}$$

Properties: - Sparse embeddings friendly - Less common than L2 - Sum of absolute values = 1

1.5.3 No Normalization

Use Cases: - When embedding model already normalizes - Dense non-normalized embeddings - Dot product metric

1.6 Embedding Model Standards

1.6.1 Sentence Transformers Format

Configuration (config_sentence_transformers.json):

```
{
  "model_name": "all-MiniLM-L6-v2",
  "max_seq_length": 256,
  "do_lower_case": true,
  "architectures": ["Transformer"],
  "model_type": "bert",
  "word_embedding_dimension": 384,
  "pooling_mode": "mean",
  "normalizer": "L2",
  "quantize": false,
  "model_card_url": "https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2",
  "architecture": {
    "0": {
      "type": "Transformer"
    },
    "1": {
      "type": "Pooling",
      "word_embedding_dimension": 384,
      "pooling_mode": "mean"
    },
    "2": {
      "type": "Normalize"
    }
  }
}
```

1.6.2 OpenAI Embedding Format

Request:

```
{
  "model": "text-embedding-3-large",
  "input": [
    "The cat sat on the mat",
    "A feline rested on textile"
  ],
  "encoding_format": "float",
  "dimensions": 1536
}
```

Response:

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [0.001, -0.023, 0.045, ...]
    },
    {
      "object": "embedding",
      "index": 1,
      "embedding": [0.002, -0.024, 0.046, ...]
    }
  ],
  "model": "text-embedding-3-large",
  "usage": {
    "prompt_tokens": 15,
    "total_tokens": 15
  }
}
```

1.7 Procedural Use-Cases

1.7.1 Use-Case 1: Generate and Store Embeddings

```
from sentence_transformers import SentenceTransformer
import numpy as np
import pickle

class EmbeddingManager:
    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        self.model = SentenceTransformer(model_name)
        self.embeddings_cache = {}

    def encode_texts(self, texts: list) -> np.ndarray:
        """
        Encode texts to embeddings

        Returns:
            embeddings: shape (n_texts, dimension)
        """
        embeddings = self.model.encode(
            texts,
            convert_to_numpy=True,
            normalize_embeddings=True
        )
        return embeddings

    def store_embeddings(self, texts: list, filepath: str):
        """Store embeddings with metadata"""
        embeddings = self.encode_texts(texts)

        data = {
            "texts": texts,
            "embeddings": embeddings,
            "model_name": self.model.get_sentence_embedding_dimension(),
            "dimension": embeddings.shape[1],
            "normalization": "L2"
        }

        with open(filepath, "wb") as f:
            pickle.dump(data, f)

    def load_embeddings(self, filepath: str) -> dict:
        """Load stored embeddings"""
        with open(filepath, "rb") as f:
            return pickle.load(f)

# Usage
manager = EmbeddingManager()
texts = ["Hello world", "How are you?", "Great day"]
manager.store_embeddings(texts, "embeddings.pkl")
```

1.7.2 Use-Case 2: Semantic Search

```

import numpy as np
from typing import List, Tuple

def semantic_search(
    query: str,
    corpus: List[str],
    model,
    top_k: int = 5
) -> List[Tuple[str, float]]:
    """Find most similar texts to query"""

    # Encode query and corpus
    query_embedding = model.encode(query, normalize_embeddings=True)
    corpus_embeddings = model.encode(corpus, normalize_embeddings=True)

    # Compute cosine similarities (dot product for normalized)
    similarities = np.dot(corpus_embeddings, query_embedding)

    # Get top-k
    top_indices = np.argsort(similarities)[::-1][:top_k]

    results = [
        (corpus[i], similarities[i])
        for i in top_indices
    ]

    return results

# Usage
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")

corpus = [
    "The quick brown fox jumps over the lazy dog",
    "A fast red fox leaps over a sleepy dog",
    "The weather is sunny today",
    "It's raining outside"
]

results = semantic_search(
    "quick fox jumps",
    corpus,
    model,
    top_k=2
)

for text, score in results:
    print(f"score: {score:.4f}: {text}")

```

1.7.3 Use-Case 3: Batch Distance Computation


```

def cosine_similarity_matrix(
    embeddings1: np.ndarray,
    embeddings2: np.ndarray
) -> np.ndarray:
    """
    Compute cosine similarity matrix for normalized embeddings

    Args:
        embeddings1: shape (n, d)
        embeddings2: shape (m, d)

    Returns:
        similarity_matrix: shape (n, m)
    """
    # Efficient batch computation using BLAS
    return np.dot(embeddings1, embeddings2.T)

# Example
embeddings1 = np.random.randn(100, 768)
embeddings2 = np.random.randn(1000, 768)

# Normalize
embeddings1 /= np.linalg.norm(embeddings1, axis=1, keepdims=True)
embeddings2 /= np.linalg.norm(embeddings2, axis=1, keepdims=True)

# Compute similarities
similarities = cosine_similarity_matrix(embeddings1, embeddings2)
print(f"Similarity matrix shape: {similarities.shape}") # (100, 1000)

# Get top-k per query
top_k = 5
top_indices = np.argsort(similarities, axis=1)[: , -top_k:]

```

1.8 Examples

1.8.1 Example 1: Embedding Storage Format

HDF5 Format (efficient for large-scale):

```

import h5py
import numpy as np

# Write embeddings
with h5py.File("embeddings.h5", "w") as f:
    texts = ["hello", "world", "test"]
    embeddings = np.random.randn(3, 768).astype(np.float32)

    f.create_dataset("embeddings", data=embeddings, compression="gzip")
    f.create_dataset("texts", data=texts, compression="gzip")
    f.attrs["model"] = "all-MiniLM-L6-v2"
    f.attrs["dimension"] = 768
    f.attrs["normalized"] = True

# Read embeddings
with h5py.File("embeddings.h5", "r") as f:
    embeddings = f["embeddings"][:]
    texts = f["texts"][:]
    model_name = f.attrs["model"]

```

1.8.2 Example 2: Quantized Embeddings

Int8 Quantization:

```
def quantize_embedding(embedding: np.ndarray) -> np.ndarray:
    """Convert float32 to int8"""
    # Assuming normalized embedding in [-1, 1]
    quantized = np.round(embedding * 127).astype(np.int8)
    return quantized

def dequantize_embedding(quantized: np.ndarray) -> np.ndarray:
    """Convert int8 back to float32"""
    embedding = quantized.astype(np.float32) / 127.0
    return embedding

# Example
original = np.array([0.5, -0.3, 0.8], dtype=np.float32)
quantized = quantize_embedding(original)
recovered = dequantize_embedding(quantized)

print(f"Original: {original}")
print(f"Quantized: {quantized}")
print(f"Recovered: {recovered}")
```

1.8.3 Example 3: Model Card for Embeddings

```
# all-MiniLM-L6-v2

## Model Details

- **Embedding Dimension**: 384
- **Max Sequence Length**: 256
- **Pooling**: Mean pooling
- **Normalization**: L2
- **Training Data**: 1B sentence pairs (MS MARCO, AllNLI)

## Performance

| Benchmark | Score |
|-----|-----|
| MTEB | 0.73 |
| STS Benchmark | 0.85 |
| Semantic Textual Similarity | 0.84 |

## Usage

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
embedding = model.encode("Hello world")
Returns shape (384,)
```

## 1.9 Distance Metrics

- **Recommended:** Cosine similarity
- **Alternative:** Euclidean distance (for normalized vectors, equivalent to cosine)
- **Output Range:** [-1, 1] (cosine) or [0, 2] (euclidean) ``